StockGuard AI

A Predictive Business Intelligence System for SMEs



Muhammad Ahsan Qasim (SP-2023-BSCS-041)
Arslan Ahmad (SP-2023-BSCS-043)

Supervised By
Miss Tayyaba Sultana

Submitted for the partial fulfillment of the BS Computer Science degree
to the Faculty of Computer Science

LAHORE GARRISON UNIVERSITY
October 2026

ABSTRACT

Small and medium-sized enterprises frequently manage inventory with spreadsheets, manual estimates and limited analytical support. These practices make it difficult to anticipate product-level demand, determine appropriate reorder quantities and distinguish temporary fluctuations from persistent change. The resulting stockouts, excess inventory and weak revenue planning can directly affect cash flow and customer service. StockGuard AI addresses this problem through a cross-platform predictive business intelligence system designed around the operational needs of SMEs.

The implemented solution combines a React web application, a React Native mobile application, a Node.js and Express business API, MongoDB Atlas and a Python forecasting service. Historical sales uploaded through validated CSV files are transformed into continuous daily time series. Linear Regression, ARIMA and Random Forest candidates are evaluated with MAE and RMSE, while an imported global XGBoost model and a seasonal-naive fallback provide additional production forecasting paths. Forecasts are available for 7, 14, 28 and 30 days and store every daily prediction, the selected model, accuracy metrics, total demand and estimated revenue.

StockGuard AI converts forecasts into inventory decisions by calculating lead-time demand, demand variability, safety stock, reorder points, target stock, days of cover and recommended order quantities. It generates low-stock, critical-stock, overstock and demand-spike recommendations and tracks each recommendation through open, approved, dismissed and completed states. The system also implements role-based access, tenant isolation, audit logs, in-app and push-notification pipelines, reports, encrypted daily backups and isolated restore verification.

The production XGBoost bundle was trained on 8,172,438 rows and evaluated through a three-fold recursive backtest containing 986,433 observations. It achieved MAE 0.574939 and RMSE 2.021802, compared with MAE 0.658729 and RMSE 2.459768 for the seasonal baseline, a 12.72 percent MAE improvement. A production load test with 100 authenticated virtual users returned 100 successful responses, an average of 259 ms and p95 of 832 ms. The completed prototype demonstrates a practical path from historical sales to a defensible replenishment decision in one secure web and mobile platform.

Keywords: Demand forecasting, inventory management, business intelligence, XGBoost, ARIMA, Random Forest, Linear Regression, SMEs, MERN, replenishment.

CERTIFICATE

It is certified that the project report titled “StockGuard AI: A Predictive Business Intelligence System for SMEs,” submitted by Muhammad Ahsan Qasim and Arslan Ahmad for partial fulfillment of the requirements of the Bachelor of Science in Computer Science, is presented for internal defence and evaluation.

The project has been completed under the supervision of Miss Tayyaba Sultana in the Department of Computer Science, Lahore Garrison University. Committee members may record their approval and date below.

Name / Designation	**Signature / Date**
Head of Department	_____
Final Year Project Coordinator	_____
Miss Tayyaba Sultana — Supervisor	_____

COPYRIGHTS

This report documents the original academic and implementation work completed for StockGuard AI by Muhammad Ahsan Qasim and Arslan Ahmad during their BS Computer Science programme at Lahore Garrison University.

No part of the project may be reproduced for academic credit by another party without appropriate acknowledgement. Third-party libraries, datasets and published methods remain subject to their respective licences and citations.

Name / Designation	**Signature / Date**
Muhammad Ahsan Qasim	_____
Arslan Ahmad	_____

DECLARATION

We declare that this dissertation and the accompanying software prototype are our own work except where sources, libraries and datasets are explicitly acknowledged. The reported implementation evidence and test results were produced from the StockGuard AI repository and its isolated production workspace.

We understand that plagiarism, fabricated evidence or unacknowledged reuse may result in academic penalties under university policy.

Name / Designation	**Signature / Date**
Muhammad Ahsan Qasim	_____
Arslan Ahmad	_____

PLAGIARISM CERTIFICATE

This is to certify that the report titled “StockGuard AI: A Predictive Business Intelligence System for SMEs” is submitted for the award of the Bachelor of Science in Computer Science and was prepared under academic supervision.

The official similarity report and any required departmental plagiarism documentation shall be attached by the students or department before final submission.

Name / Designation	**Signature / Date**
Miss Tayyaba Sultana — Supervisor	_____

ACKNOWLEDGMENT

We are grateful to our supervisor, Miss Tayyaba Sultana, for her guidance, careful review and continued encouragement throughout the design, development and verification of StockGuard AI. Her feedback helped us connect the technical implementation with the practical problems faced by small and medium-sized businesses.

We also thank the Department of Computer Science and the Final Year Project committee at Lahore Garrison University for providing the academic framework for this work. We acknowledge the open-source communities behind React, Node.js, MongoDB, Python, scikit-learn, Statsmodels, XGBoost, React Native and Expo, whose tools made the cross-platform prototype possible.

Finally, we thank our families, friends and classmates for their patience and support. Their encouragement was particularly valuable during data preparation, model training, deployment and the final verification phase.

Table of Contents

ABSTRACT 2
ACKNOWLEDGMENT 7
Table of Contents 8
List of Figures 11
List of Tables 12
INTRODUCTION 13
1.1 Background 13
1.2 Project Motivation 13
1.3 Problem Statement 13
1.4 Aim and Objectives 13
1.5 Research Questions 14
1.6 Significance of the Study 14
1.7 Scope and Delimitations 14
1.8 Structure of the Report 14
LITERATURE REVIEW 15
2.1 Introduction 15
2.2 Linear Regression 15
2.3 ARIMA 15
2.4 Random Forest and Gradient-Boosted Trees 15
2.5 Model Evaluation 15
2.6 Forecast-Based Inventory Control 16
2.7 Business Intelligence and Visualisation 16
2.8 Security and Multi-Tenant SaaS 16
2.9 Research Gap 16
2.10 Chapter Summary 17
PROBLEM STATEMENT AND PROPOSED FRAMEWORK 18
3.1 Formal Problem Definition 18
3.2 Stakeholders and User Classes 18
3.3 Delivered Functional Areas 18
3.4 Proposed System Architecture 18
3.5 Data Model 19
3.6 End-to-End Workflow 19
3.7 Role and Permission Structure 19
3.8 Constraints and Assumptions 20
METHODOLOGY AND IMPLEMENTATION 21
4.1 Development and Research Method 21
4.2 Dataset and Feature Dataset 21

4.3 Data Validation and Preprocessing 21
4.4 Feature Engineering 21
4.5 Forecasting Algorithms 22
4.5.1 Linear Regression 22
4.5.2 ARIMA 22
4.5.3 Random Forest 22
4.5.4 Global XGBoost and Seasonal Fallback 22
4.6 Validation and Model Selection 22
4.7 Forecast Request Pipeline 22
4.8 Revenue and Replenishment Calculations 23
4.9 Software Implementation 23
4.10 Security Implementation 23
4.11 Notifications and Reports 24
4.12 Backup, Monitoring and Recovery 24
4.13 Testing Strategy 24
4.14 Chapter Summary 25
RESULTS AND DISCUSSION 26
5.1 Model Training and Backtest Results 26
5.2 Live Candidate Comparison 26
5.3 Functional Verification 27
5.4 Performance Evaluation 27
5.5 Recovery Verification 27
5.6 Presentation Workspace 28
5.7 Requirements Compliance 28
5.8 Discussion 28
5.9 Limitations 28
SUMMARY, CONTRIBUTIONS AND CONCLUSION 29
6.1 Summary 29
6.2 Main Findings 29
6.3 Contributions 29
6.4 Recommendations 29
6.5 Future Work 29
6.6 Conclusion 30
REFERENCES 31
SRS REQUIREMENT COMPLIANCE 32
VERIFICATION EVIDENCE 33
PRINCIPAL REST API ENDPOINTS 34
INTERNAL DEFENCE DEMONSTRATION SEQUENCE 35
Defence Evidence Summary 35

SUBMISSION AND DEPLOYMENT CHECKLIST 36

List of Figures

Figure 1: StockGuard AI system architecture

Figure 2: Product-level forecasting workflow

Figure 3: Replenishment decision flow

Figure 4: Three-fold recursive XGBoost backtest

Figure 5: Verified production performance

Figure 6: Deployment, monitoring and recovery workflow

List of Tables

Table 1: Project objectives and completion evidence

Table 2: Forecasting model comparison

Table 3: Research gap and StockGuard response

Table 4: Delivered functional areas

Table 5: Core MongoDB entities

Table 6: Role and permission structure

Table 7: Feature engineering variables

Table 8: Technology stack

Table 9: Three-fold recursive backtest results

Table 10: Functional verification summary

Table 11: SRS requirement compliance

Table 12: Principal REST API endpoints

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized enterprises contribute employment, innovation and local economic activity, yet many operate without specialised forecasting or inventory-planning teams. Sales history is commonly stored in spreadsheets or point-of-sale exports, while replenishment decisions depend on recent experience and informal judgement. Such methods may be adequate for a stable product, but they become unreliable when a business manages multiple stores, seasonal items and products with different selling rates.

Inventory decisions involve a direct trade-off. Insufficient stock produces lost sales, delayed orders and dissatisfied customers. Excess stock ties up working capital, occupies storage space and may become obsolete. The decision is more difficult because demand changes over time and because the business must order before future demand becomes observable. A useful system therefore needs to estimate demand, communicate uncertainty and translate the estimate into a concrete action.

Modern data analytics makes it possible to learn patterns from historical transactions. Statistical models can represent trend and seasonality, while machine-learning models can capture nonlinear relationships among lagged demand, rolling averages and calendar variables. A business intelligence layer can then convert predictions into revenue, stock-risk and replenishment information. StockGuard AI brings these functions together in a SaaS-style application intended for non-technical SME users.

1.2 Project Motivation

The proposal identified that enterprise forecasting products are often costly, difficult to configure or too broad for small organisations. An SME needs a focused workflow: create a product catalogue, upload historical sales, request a product-level forecast, compare models and decide how much to order. The solution should work in a browser and on a mobile device, support multiple users and protect each business workspace from other tenants.

A second motivation was explainability at the operational level. Forecast accuracy alone does not tell a purchasing officer what to do. StockGuard AI therefore combines the selected forecast with supplier lead time, current and reserved inventory, historical variability and a configurable safety-stock policy. The system explains why it recommends an order, highlights urgency and records the action lifecycle.

1.3 Problem Statement

SMEs possess sales data but frequently lack an affordable, secure and understandable system for transforming it into product-level demand forecasts and replenishment decisions. Manual estimates and disconnected spreadsheets can produce misleading forecasts, inconsistent stock policies, weak revenue planning and delayed responses to low stock or demand spikes. Existing tools may omit product-level model comparison, cross-platform access, multi-user control or direct stock recommendations.

The project problem is therefore to design and implement a cross-platform predictive business intelligence system that accepts validated sales histories, compares appropriate forecasting models, stores complete future predictions, converts those predictions into business actions and provides the result to authorized users within the performance and security limits stated in the SRS.

1.4 Aim and Objectives

The main aim is to provide SMEs with a practical forecasting and inventory decision-support platform that is accurate enough for product-level planning, simple enough for routine use and secure enough for multi-user business data.

Table 1: Project objectives and completion evidence

Objective	Delivered evidence

Objective	Delivered evidence
Generate demand forecasts	Product and store series support 7, 14, 28 and 30-day horizons.
Compare forecasting candidates	Linear Regression, ARIMA, Random Forest, XGBoost and seasonal fallback paths expose MAE/RMSE evidence.
Support inventory decisions	Lead-time demand, variability, safety stock, target stock and order quantity are calculated.
Deliver web and mobile access	React/Vite and React Native/Expo clients share the protected business API.
Protect multi-user data	JWT sessions, role permissions, tenant filters and audit logs are implemented.
Verify operational quality	Automated checks, load tests, health monitoring and encrypted recovery evidence are included.

1.5 Research Questions

1. How can multiple forecasting methods be compared fairly on the same product-level history?
2. How can a forecast be converted into a repeatable stock-replenishment recommendation?
3. How can SME users access analytical functions through web and mobile interfaces without losing security or tenant isolation?
4. Can the deployed prototype satisfy the response-time, concurrency, forecasting and recovery targets in the submitted SRS?

1.6 Significance of the Study

The project demonstrates applied integration rather than a single isolated model. It connects data validation, time-series modelling, inventory logic, access control, visualisation and deployment in one working system. This matters because an accurate notebook experiment is not automatically an operational product. The implemented API, persistence rules, user workflows and recovery controls show how predictive analytics can become a business process.

For SMEs, the practical contribution is a clear path from historical sales to an order decision. For software engineering, the project demonstrates modular separation between user interfaces, business services, forecasting services and storage. For academic evaluation, the report provides traceable evidence between the proposal, SRS requirements, implementation and measured results.

1.7 Scope and Delimitations

The delivered scope includes registration and authentication; role-based access; profile and settings management; product, store, inventory and sales records; CSV upload and validation; demand and revenue forecasting; model comparison; replenishment recommendations; dashboards, reports and insights; notifications; administration; audit logging; web and mobile clients; production deployment; monitoring; encrypted backup and restore verification.

The prototype is limited to historical business sales and configured inventory inputs. It does not automatically ingest point-of-sale streams, supplier portals, promotions, weather or macroeconomic data. SMTP credentials and platform store credentials are external operational settings. Forecast quality remains dependent on the continuity, relevance and accuracy of each business dataset.

1.8 Structure of the Report

Chapter 2 reviews forecasting, evaluation, inventory and business intelligence concepts. Chapter 3 defines the problem and proposed framework. Chapter 4 explains the methodology and implementation. Chapter 5 presents model, functional, performance and recovery results. Chapter 6 summarises contributions, recommendations, limitations, future work and the conclusion. Appendices provide requirements, tests, API routes and a defence demonstration sequence.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

Demand forecasting estimates future sales from historical observations and relevant explanatory variables. In inventory management, the forecast supports decisions made before demand occurs. The literature includes statistical time-series methods, supervised machine-learning methods and ensemble models. Their usefulness depends on the data pattern, history length, forecast horizon, computational cost and ability to generalise.

2.2 Linear Regression

Linear Regression represents the target as a weighted combination of input variables. For product demand, the variables may include lags, rolling averages and cyclical calendar features. The method is fast, reproducible and easy to inspect, which makes it a valuable baseline. Its central limitation is that a fixed linear relationship cannot naturally represent every nonlinear demand pattern or interaction.

StockGuard uses Linear Regression as one candidate in the Python service. The candidate receives the same prepared history and validation split as the other methods. This prevents a favourable data split from giving one algorithm an artificial advantage and allows its MAE and RMSE to be interpreted beside other models.

2.3 ARIMA

Autoregressive Integrated Moving Average models combine autoregression, differencing and moving-average error terms. They are established tools for univariate time-series forecasting and can represent temporal dependence without requiring a large feature matrix. Their assumptions and parameter selection require care, particularly when a series is short, intermittent or changes structure.

ARIMA is included because some sales series are primarily driven by their own recent values and temporal structure. In StockGuard, it is evaluated at request time for the selected product/store series. If the model cannot be estimated reliably for a short or unsuitable series, the service returns a controlled candidate error and the comparison continues with the remaining models.

2.4 Random Forest and Gradient-Boosted Trees

Random Forest combines multiple decision trees trained on resampled data and random feature subsets. Averaging reduces the variance of a single tree and makes the method effective for nonlinear relationships. It can use lag, rolling and calendar variables without requiring a predetermined linear form. Its predictions remain dependent on patterns represented in the training history and may be conservative beyond the observed range.

Gradient-boosted trees build an additive sequence of learners, with each stage focusing on the remaining error. XGBoost provides regularisation, efficient tree construction and portable model serialization. StockGuard extends the three proposal models with a verified global XGBoost bundle trained on millions of store/product/day observations. This production candidate is used where category mappings and sufficient features are available; otherwise the candidate pipeline can use the seasonal fallback and Python models.

2.5 Model Evaluation

Forecast evaluation must preserve temporal order. Randomly shuffling observations can expose future information to the training set and overstate accuracy. StockGuard uses holdout periods and recursive forecasting: a model is trained on observations available before a cutoff, predicts the future period, and is compared with the actual values. Repeating this process at several cutoffs measures stability across time.

Mean Absolute Error (MAE) is the average absolute difference between actual and predicted demand. Root Mean Squared Error (RMSE) takes the square root of the mean squared error and penalises large mistakes more heavily. The application selects the candidate with the lowest MAE and uses RMSE as a tie-breaker. Both measures are retained so users can distinguish consistent small errors from occasional large deviations.

Table 2: Forecasting model comparison

Model	Strength	Limitation	Role in StockGuard
Linear Regression	Fast and interpretable baseline	Limited nonlinear representation	Per-series candidate
ARIMA	Explicit temporal dependence	Sensitive to short or unstable series	Per-series time-series candidate
Random Forest	Nonlinear relationships and interactions	Can be conservative outside observed range	Per-series ML candidate
Global XGBoost	Scalable tree ensemble with learned cross-series patterns	Requires matching feature/category mappings	Verified production candidate
Seasonal naive	Robust and transparent fallback	Repeats past seasonal behaviour	Fallback and benchmark

2.6 Forecast-Based Inventory Control

A demand forecast becomes operational when it is linked with lead time and stock position. Lead-time demand estimates the units expected before a replenishment order arrives. Safety stock protects against variability and forecast error. The reorder point identifies the stock level at which an order is needed, while target stock represents the desired inventory after replenishment.

Textbook policies often use a service-level factor multiplied by demand standard deviation and the square root of lead time. Practical businesses may also define a minimum safety-stock percentage. StockGuard calculates both a variability-based amount and a policy amount, then uses the larger. This makes the recommendation responsive to observed volatility while respecting the business setting.

2.7 Business Intelligence and Visualisation

Business intelligence combines data preparation, analysis and presentation to support decisions. Forecasts are easier to act on when users can see recent history, future daily values, revenue implications, category behaviour and stock risk in the same workspace. Interactive dashboards reduce the effort needed to inspect a single product, while reports create a shareable record of the analysis.

A useful interface must distinguish an estimate from an action. StockGuard therefore separates forecast outputs from recommendation status. Users can review the calculation, approve an action, dismiss it when business context contradicts the model, or mark it completed after purchasing. Audit events make these decisions reviewable.

2.8 Security and Multi-Tenant SaaS

A multi-tenant SaaS application stores records for several organisations in shared infrastructure. Correct isolation depends on applying the authenticated workspace identifier to every business query, not merely hiding data in the interface. Authentication, authorization and tenant filtering are separate controls: a valid user must still possess permission for the route, and the query must still be restricted to the user's business.

StockGuard applies bcrypt password hashing, short-lived JWT access tokens, revocable refresh sessions, role/permission middleware and tenant-scoped Mongoose queries. Web refresh credentials use an HttpOnly cookie, whereas the mobile client stores its refresh token in Expo SecureStore. Password changes and resets rotate a session version so previously issued sessions become invalid.

2.9 Research Gap

The reviewed approaches often solve one layer of the problem: a notebook compares models, an inventory package records quantities, or a dashboard visualises past sales. The project gap is an affordable cross-platform workflow that connects validated SME data to model comparison, complete 30-day predictions, revenue estimates, replenishment actions, role-controlled collaboration, notifications and evidence-based recovery.

Table 3: Research gap and StockGuard response

Observed gap	StockGuard AI response
Manual estimates and disconnected spreadsheets	Validated upload, persistent product/store series and repeatable forecast runs
One model used for every series	LR, ARIMA, RF, XGBoost and seasonal candidates with MAE/RMSE selection
Forecast without a purchasing action	Safety stock, target stock, order quantity and recommendation lifecycle
Desktop-only or analyst-only tools	Responsive web and React Native mobile application

Observed gap	StockGuard AI response
Weak separation between businesses	JWT, RBAC, permission checks and tenant-scoped queries
No operational recovery evidence	Encrypted scheduled backups and isolated restore verification

2.10 Chapter Summary

This chapter reviewed the five forecasting paths used by StockGuard AI, the temporal validation approach, the role of MAE and RMSE, and the inventory concepts required to convert predicted demand into a purchasing decision. Linear Regression offers an interpretable baseline, ARIMA captures univariate time dependence, Random Forest represents nonlinear relationships, XGBoost provides a verified global ensemble, and seasonal naive supplies a robust fallback.

The review also established that forecasting accuracy, application usability and data protection must be considered together. The identified research gap is an SME-oriented system that connects model comparison with complete forecast persistence, replenishment recommendations, role-controlled collaboration and operational recovery. Chapter 3 translates that gap into the proposed StockGuard framework.

CHAPTER 3
PROBLEM STATEMENT AND PROPOSED FRAMEWORK

3.1 Formal Problem Definition

Given a sequence of dated sales observations for a product at a store, current and reserved inventory, unit price, supplier lead time and workspace policy settings, StockGuard must estimate future daily demand for a requested horizon. It must compare eligible candidates on a historical validation window, select the candidate with the lowest measured error, store the selected predictions and convert the forecast into revenue and replenishment information.

The system must provide this function only to authorized members of the owning business, support up to 10,000 uploaded rows and a 10 MB CSV, display meaningful validation errors, maintain audit history, and operate on contemporary web, Android and iOS clients. The application must remain usable when an optional integration such as SMTP is not configured.

3.2 Stakeholders and User Classes

The principal stakeholder is the SME business owner who requires a consolidated view of products, sales, forecast risk and purchasing actions. Administrators manage users, invitations, roles, permissions and system activity. Managers supervise products, inventory and recommendations. Analysts explore forecasts, reports and insights. Staff members perform permitted operational tasks. The project team and system operators maintain deployment, monitoring, backups and model services.

3.3 Delivered Functional Areas

Table 4: Delivered functional areas

Area	Implemented capability
Identity	Registration, login, refresh, logout, password change/reset, profile
Products and inventory	Product CRUD, stores, stock quantities, lead time and pricing
Data ingestion	CSV header/type/date/duplicate checks, idempotent import and rollback
Forecasting	7/14/28/30-day candidate comparison and saved daily predictions
Decision support	Revenue, reorder point, safety stock, target stock and order quantity
Business intelligence	Dashboard, reports, insights and category/product analytics
Administration	Invitations, users, suspension, roles, permissions and audit logs
Notifications	Upload, forecast, low-stock, critical-stock and recommendation events
Operations	Health checks, encrypted backup, restore verification and deployment profiles

3.4 Proposed System Architecture

The framework uses a layered architecture so that user interfaces, business rules, model execution and persistence can evolve independently. The React web client and React Native mobile client communicate with the same Express API. The API authenticates the request, validates data, enforces permissions and business isolation, orchestrates forecasting services and persists results in MongoDB Atlas.

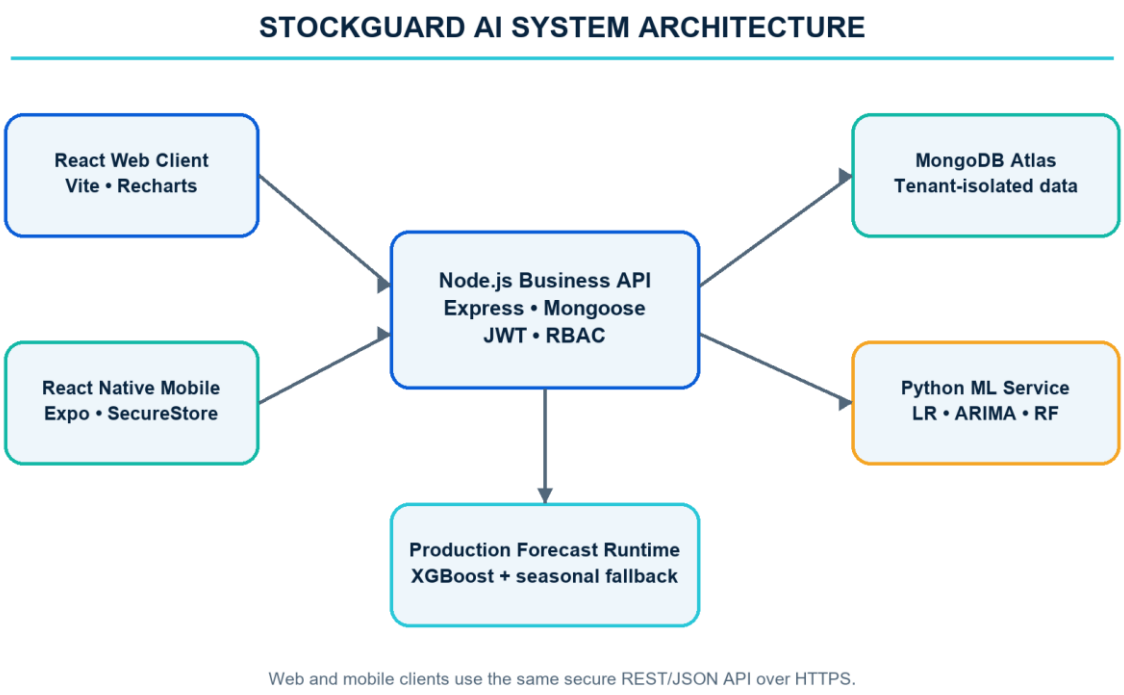


Figure 1: StockGuard AI system architecture

The Python service provides Linear Regression, ARIMA and Random Forest candidates. The Node runtime loads the verified XGBoost JSON model and its category mappings and provides a seasonal fallback. The API normalizes candidate results, selects the best valid model and applies decision-support logic. Vercel hosts the production web/API functions, while GitHub Actions performs scheduled health and recovery tasks. Expo Application Services provides mobile build infrastructure.

3.5 Data Model

MongoDB collections are organised around a Business tenant. Each operational document stores a business identifier, and service methods include that identifier in queries and writes. This pattern supports shared infrastructure without mixing business records.

Table 5: Core MongoDB entities

Entity	Purpose	Important relationships
Business	Workspace settings and defaults	Owns users, stores, products and all analytical records
User	Identity, role, permissions and session version	Belongs to one business
Store	Business location/code	Owns inventory and sales series
Product	SKU, category, price and supplier lead time	Referenced by sales, inventory and forecasts
Sale	Dated quantity and revenue	Links business, store and product
ForecastRun / Forecast	Model metrics, totals and daily predictions	Links series and selected candidate
Inventory	On-hand and reserved stock	Unique by product/store/workspace
Recommendation	Risk, quantity, reason and lifecycle	Derived from a forecast and inventory plan
Notification	Persistent user alert	May refer to upload, forecast or recommendation
AuditLog	Security and activity event	Records actor, action, metadata and timestamp

3.6 End-to-End Workflow

An authorized user creates or selects a product and store, then uploads historical sales in the required CSV format. The API validates the file before committing records. A forecast request identifies the product, store and horizon. The API loads the series and inventory context, invokes eligible candidates and stores the chosen result. Decision support calculates replenishment measures and creates or updates recommendations. The clients then display the forecast, recommendations, reports and alerts.

3.7 Role and Permission Structure

Table 6: Role and permission structure

Role	Typical access
Owner	Full workspace, user, security, configuration and analytical access
Administrator	User management, audit logs, products, uploads, forecasts and reports
Manager	Products, inventory, forecasts, recommendations and reports
Analyst	Uploads, model comparison, forecasts, reports and insights
Staff	Assigned operational read/write actions only

Roles provide sensible defaults, while custom permission arrays allow an owner or administrator to adjust access for a specific user. Backend middleware remains the authority; hiding a navigation item in the client is only a usability measure and does not grant or revoke API access.

3.8 Constraints and Assumptions

- Forecasts assume that the uploaded product/store history is correctly dated and represents the business process being forecast.
- Products with insufficient history may require the seasonal fallback or may produce wider practical uncertainty.
- The XGBoost candidate requires the imported category mappings and the twelve training features used by the model bundle.
- Email delivery depends on external SMTP credentials; in-app notifications remain available without SMTP.
- Physical iOS and Android distribution depends on Apple/Google developer credentials outside the source repository.
- Cloud availability depends on Vercel, MongoDB Atlas and the configured Python service, with health checks used to detect failures.

CHAPTER 4
METHODOLOGY AND IMPLEMENTATION

4.1 Development and Research Method

The project followed an iterative engineering method. Requirements were extracted from the approved proposal and SRS, converted into implementation phases, and verified against working endpoints and interfaces. Forecasting work proceeded from data profiling to feature generation, baseline evaluation, recursive backtesting, production model packaging and application integration. Application work proceeded from authentication and database schemas to business workflows, mobile parity, operational controls and final evidence.

Each phase produced a reviewable artifact: a schema, endpoint, interface screen, model bundle, automated assertion or production result. This reduced the risk of completing attractive interfaces without functioning backend behaviour. The final compliance matrix maps all eighteen functional requirements to implementation evidence.

4.2 Dataset and Feature Dataset

The model training dataset contains daily retail sales observations covering 9 August 2017 to 31 October 2019. Feature generation produced 8,924,278 rows across 142 stores and 414 products. Categorical identifiers were converted through stored mappings so the same codes can be used when the serialized model is loaded for inference.

The final production training matrix used 8,172,438 rows after the historical windows needed for lag and rolling variables were available. The trained global XGBoost bundle contains the model JSON, category mappings, registry data, saved future forecasts and a manifest with the exact XGBoost version and SHA-256 checksum.

4.3 Data Validation and Preprocessing

The upload service accepts CSV files up to 10 MB and limits a single import to 10,000 rows. Required headers are checked before processing. Dates are parsed into a standard representation, quantity and revenue fields are validated as numbers, duplicates are detected, and product/store references are resolved within the authenticated workspace. The import is idempotent and uses rollback behaviour so a failed batch does not leave a partial dataset.

For forecasting, sales are aggregated into a continuous daily series. Missing calendar dates are represented with zero demand where appropriate rather than being silently removed. This preserves the real interval between observations and prevents a seven-day lag from meaning seven recorded rows of an irregular series.

4.4 Feature Engineering

Table 7: Feature engineering variables

Feature	Purpose
store_code	Encoded store identity for the global model
product_code	Encoded product identity for the global model
lag_1	Demand from the previous day
lag_7	Demand from the same weekday one week earlier
lag_14	Demand two weeks earlier
lag_28	Four-week seasonal reference
rolling_7	Short-term average demand
rolling_28	Longer recent baseline
day_sin / day_cos	Cyclical weekday representation without an artificial boundary
month_sin / month_cos	Cyclical annual position

Recursive multi-day forecasting feeds each generated prediction into the future history used to construct the next day's lag and rolling values. This is stricter than one-step evaluation because errors can propagate across the horizon, which matches how a real 30-day forecast is consumed.

4.5 Forecasting Algorithms

4.5.1 Linear Regression

The Linear Regression candidate uses engineered lag, rolling and calendar variables to fit a continuous demand target. It provides a fast baseline and helps detect when a complex model adds little value. Negative predictions are clipped to zero because product demand cannot be negative.

4.5.2 ARIMA

The ARIMA candidate models the product/store series directly. The service validates that sufficient observations exist, fits a controlled order, generates the requested horizon and reports a candidate-level error if estimation is not possible. A failed candidate does not abort the entire forecast request.

4.5.3 Random Forest

The Random Forest candidate uses the same engineered supervised-learning matrix as Linear Regression. Multiple decision trees model nonlinear thresholds and interactions, while ensemble averaging reduces the sensitivity of a single tree. A fixed random seed supports reproducible evaluation.

4.5.4 Global XGBoost and Seasonal Fallback

The global XGBoost model uses shared information across mapped stores and products. It was trained with 83 boosting rounds on GPU using XGBoost 3.2.0 and saved as a portable JSON model with a SHA-256 checksum. At production startup, the runtime verifies the manifest and model hash before inference. Series outside the mapping can use the seasonal-naive fallback and the Python candidate service.

4.6 Validation and Model Selection

The validation set follows the training period in time. MAE is computed as $(1/n)\ \Sigma|y_i - \hat{y}_i|$. RMSE is computed as $\sqrt{[(1/n)\ \Sigma(y_i - \hat{y}_i)^2]}$. The candidate with the smallest MAE is selected; RMSE is used when MAE values are tied. Candidate metadata is retained for transparency, and any invalid or failed model is excluded with its error recorded.

For the imported production model, recursive backtests used cutoffs of 9 August 2019, 6 September 2019 and 4 October 2019. Each fold trained only on earlier rows and evaluated the following 28 days. The combined result contains 986,433 observations.

4.7 Forecast Request Pipeline

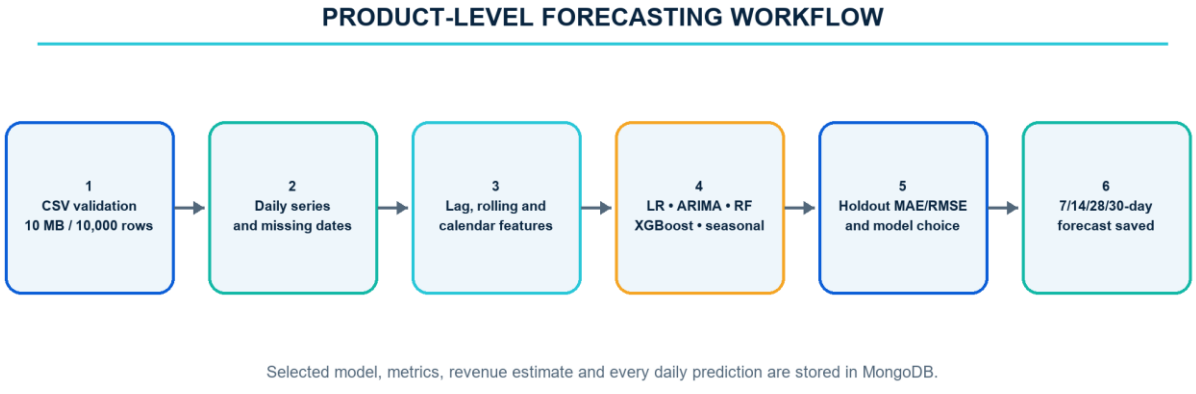


Figure 2: Product-level forecasting workflow

A successful run stores the horizon, selected model, MAE/RMSE, prediction dates and quantities, forecast units, forecast revenue and inventory plan. Storing all daily points is necessary for charts, reports, later comparison and the verified requirement that a 30-day run contains thirty predictions.

4.8 Revenue and Replenishment Calculations

Forecast revenue uses historical average unit revenue where available; otherwise the current product price is used. For inventory planning, average daily demand equals total forecast units divided by horizon days. Lead-time demand equals average daily demand multiplied by supplier lead-time days. Demand variability is the population standard deviation of historical daily quantities.

Variability safety stock equals 1.65 × demand standard deviation × √lead time. Policy safety stock equals lead-time demand × configured safety-stock percentage. The system uses the larger value. Reorder point is the ceiling of lead-time demand plus safety stock. Target stock is the ceiling of horizon forecast units plus safety stock. Recommended order quantity is max(0, target stock − available stock), where available stock is on-hand stock less reserved stock.

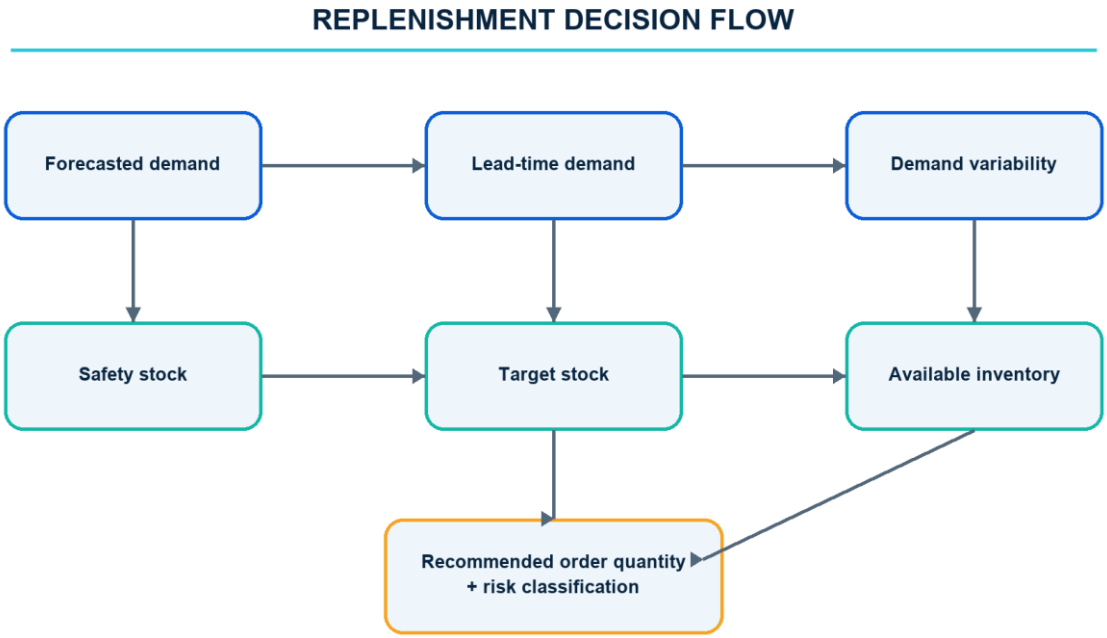


Figure 3: Replenishment decision flow

An order is high risk when available stock is zero or provides two or fewer forecast days of cover. Overstock is flagged when available inventory exceeds 125 percent of target stock. A demand spike requires forecast average demand to be at least 25 percent above the historical mean and at least one historical standard deviation higher. Duplicate open recommendations for the same business, product, store and type are prevented.

4.9 Software Implementation

Table 8: Technology stack

Layer	Technology	Implementation responsibility
Web	React, Vite, Recharts	Responsive dashboard, products, upload, forecast, reports, insights, admin and settings
Mobile	React Native, Expo	Secure cross-platform access to the same business functions
API	Node.js, Express, Zod	REST routes, validation, sessions, permissions, orchestration and errors
Persistence	MongoDB Atlas, Mongoose	Tenant business records, indexes and TTL audit retention
Python ML	Pandas, scikit-learn, Statsmodels	LR, RF and ARIMA candidate forecasts
Production ML	XGBoost JSON runtime	Verified global recursive inference and seasonal fallback
Operations	Vercel, GitHub Actions, EAS	Deployment, monitoring, backup verification and mobile builds

4.10 Security Implementation

Passwords are hashed with bcrypt and never returned by the API. Access tokens are short lived. Refresh sessions are revocable and include a session version. The web client receives its refresh credential through an

HttpOnly cookie, reducing exposure to client-side scripts. The mobile client uses Expo SecureStore. Logout revokes the refresh session, and password change or reset invalidates prior sessions.

Route middleware checks authentication, named permissions and tenant membership. Every business-scoped model query includes the authenticated business identifier. Zod schemas validate JSON inputs, Multer limits file size, and the central error middleware returns controlled JSON messages. Audit events record logins, uploads, forecasts, profile/security changes and administrative actions with a one-year retention period.

4.11 Notifications and Reports

Persistent in-app notifications are created when an upload completes, a forecast becomes ready, stock becomes low or critical, or a high-risk recommendation is generated. Expo device registrations enable mobile push delivery when platform credentials are present. SMTP delivery is optional and uses the same event pipeline, so the application remains functional when outbound email is not configured.

Users can generate sales, inventory, forecast and replenishment reports. Report records remain tenant-scoped and can be downloaded by authorized web and mobile users. Replenishment reports include recommended quantities, risk, safety stock and status so purchasing actions can be reviewed outside the live dashboard.

4.12 Backup, Monitoring and Recovery

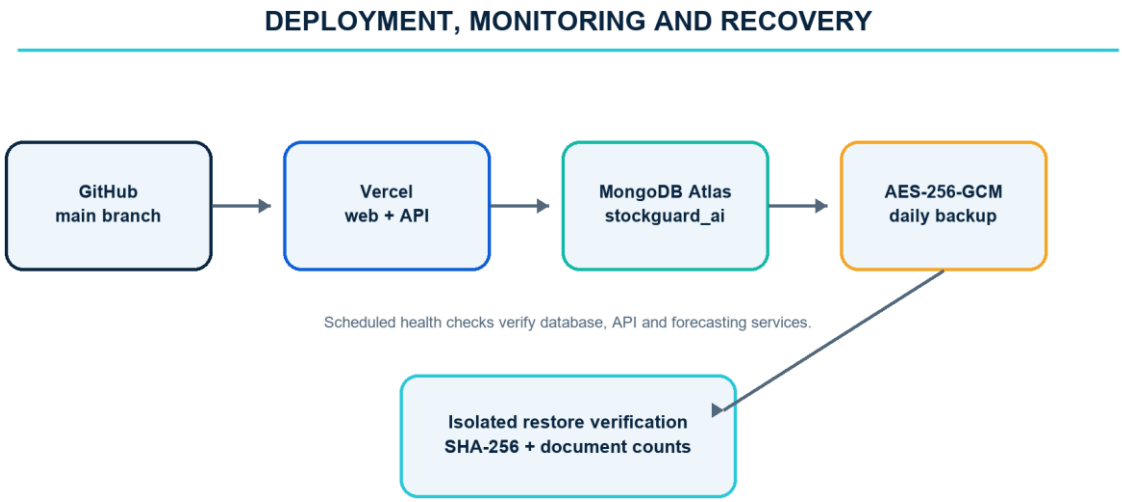


Figure 4: Deployment, monitoring and recovery workflow

A scheduled GitHub Actions workflow exports the MongoDB database, packages collection data with a manifest, encrypts it using AES-256-GCM and retains the artifact for thirty days. Restore verification decrypts the artifact, validates its hash and restores records to temporary collection names. It compares document counts and removes the temporary collections after verification, avoiding interference with live business data.

A separate scheduled health workflow checks the production API, MongoDB connectivity, Python ML status, XGBoost availability and supported horizons. These checks create operational evidence and provide early warning when an external service is unavailable.

4.13 Testing Strategy

Verification was organised at four levels. Static checks inspect route, model and permission definitions. Integration checks create isolated users and business records to test authentication, products, imports, forecasts, analytics and tenant isolation. Client builds validate web bundling and mobile TypeScript/export compatibility. Production checks exercise deployed login, dashboard, CSV validation, forecasting and concurrency thresholds.

Test data is isolated from normal business records and cleaned after verification. The dedicated presentation workspace is deterministic so the defence demonstration does not depend on last-minute data entry. Backup

restoration also uses isolated collection prefixes, which allows integrity checks without overwriting production collections.

4.14 Chapter Summary

The implemented methodology connects validated sales ingestion, continuous daily-series construction, shared feature engineering, time-ordered model evaluation and recursive multi-day forecasting. The same forecast result feeds revenue estimation and the replenishment engine, ensuring that dashboard values, reports, notifications and recommended quantities are derived from one stored analytical record.

The software layers mirror this workflow: React and React Native provide user access, Express enforces business rules and security, Python and the XGBoost runtime provide forecasting candidates, MongoDB stores tenant-scoped records, and scheduled workflows verify health and recovery. Chapter 5 evaluates the resulting model and application against recorded evidence.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Model Training and Backtest Results

The imported model bundle records 8,172,438 final training rows, 83 boosting rounds, XGBoost 3.2.0 and a 3.44 MB compressed bundle. The feature dataset covered 142 stores, 414 products and dates from 9 August 2017 to 31 October 2019. The saved manifest records the model SHA-256 value 61dda9da18aa090ed26304319c7162215f0a0158a46af18dbfcc555f92851c3c.

Table 9: Three-fold recursive backtest results

Fold	Cutoff	Observations	XGBoost MAE	XGBoost RMSE	Seasonal MAE	Seasonal RMSE
1	2019-08-09	340,765	0.588152	2.540696	0.674038	2.869917
2	2019-09-06	324,889	0.559697	1.476403	0.620590	1.852476
3	2019-10-04	320,779	0.576340	1.872189	0.681093	2.525994
Combined	Three folds	986,433	0.574939	2.021802	0.658729	2.459768

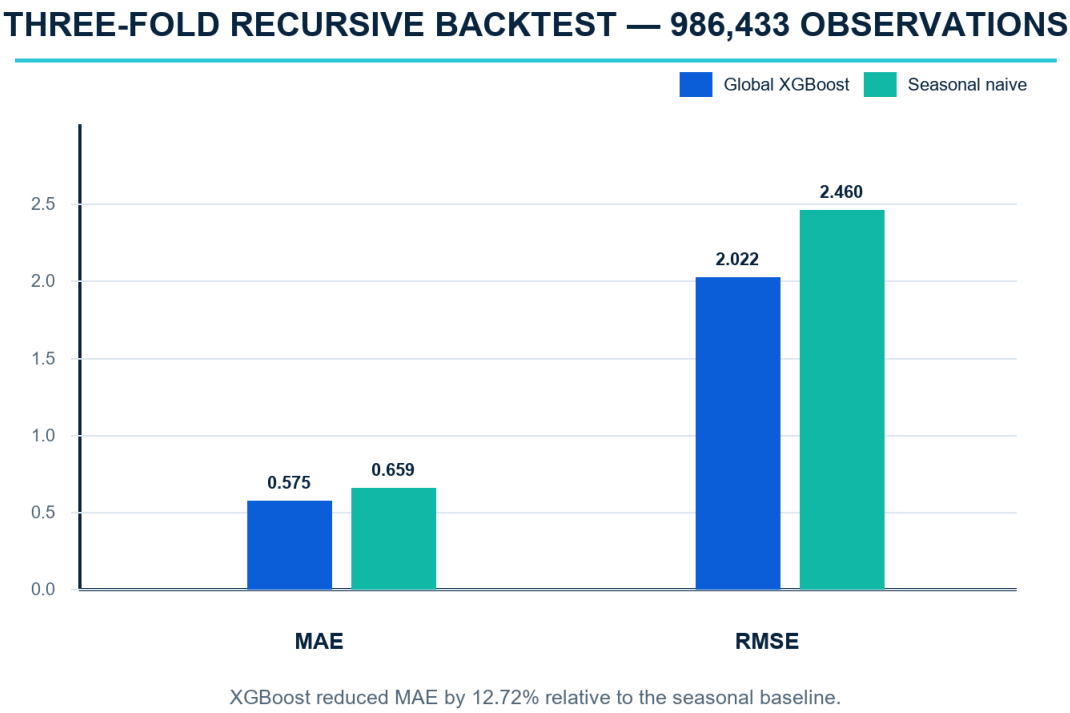


Figure 5: Three-fold recursive XGBoost backtest

The 12.72 percent MAE improvement shows that the global XGBoost model provided a more accurate central forecast than repeating the seasonal value across the tested series. RMSE also improved, indicating fewer or smaller large errors overall. Fold 1 has higher RMSE than the later folds, demonstrating why a single validation window would give an incomplete view of performance.

5.2 Live Candidate Comparison

The live application sends the selected product/store history to the Python service and obtains Linear Regression, ARIMA and Random Forest candidates with MAE and RMSE. In parallel, the Node runtime evaluates XGBoost where identifiers map to the training registry and provides a seasonal candidate. The API normalizes the candidate objects, removes failures and chooses the lowest MAE with RMSE as the tie-breaker.

This design satisfies the proposal's required comparison while preserving the separately trained production model. It also avoids claiming that one algorithm is always best. A linear or seasonal model can win for a stable series, while a tree ensemble can win for a nonlinear or irregular series. The selected model is stored with each run so the decision remains inspectable.

5.3 Functional Verification

Table 10: Functional verification summary

Area	Verified result
Authentication	Registration, login, refresh, logout, revocation and password reset passed
Account	Profile persistence, audited password change and old-session invalidation passed
Products/imports	Tenant CRUD, 10 MB CSV checks, idempotency and rollback passed
Forecasting	Python comparison, XGBoost/seasonal candidates and thirty saved daily points passed
Replenishment	Safety stock, order quantity, lifecycle and high-risk alerts passed
Analytics	Date windows, revenue, categories, low stock and tenant isolation passed
Web	Vite production build passed
Mobile	TypeScript validation and Expo Android production export passed
Dependencies	Production server audit recorded zero known vulnerabilities at verification time
Recovery	Encrypted backup codec, tamper detection and isolated restore passed

5.4 Performance Evaluation

The production performance test used 100 authenticated virtual users over a ten-second ramp. Every request returned HTTP 200. The average response time was 259 ms, p95 was 832 ms and the maximum was 1,210 ms. Login completed in 855 ms, the dashboard in 843 ms, a live 30-day forecast in 2,224 ms and validation of a 10,000-row CSV fixture in 61 ms.

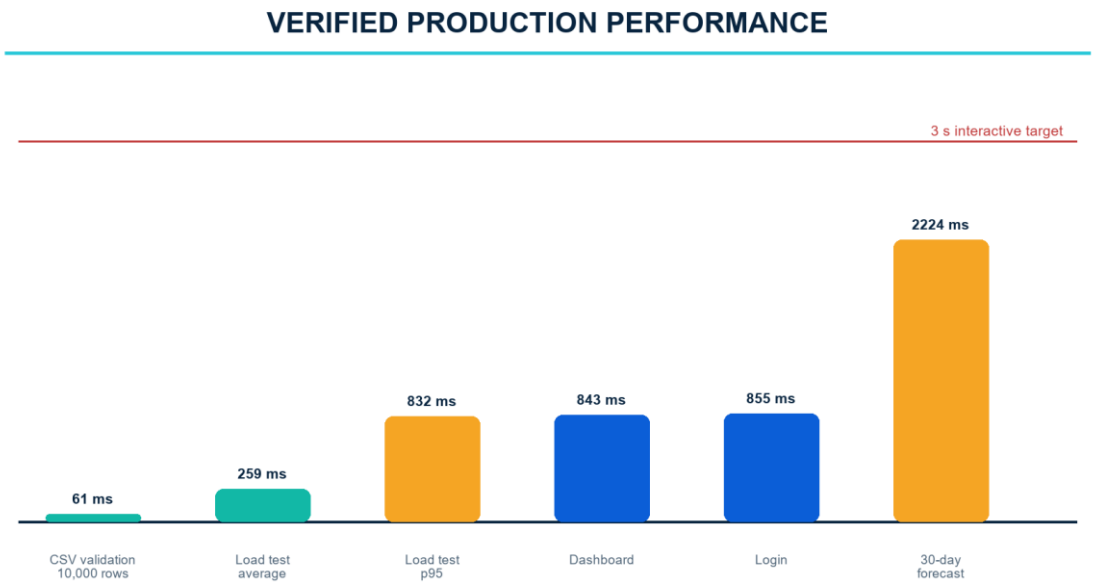


Figure 6: Verified production performance

The measured values satisfy the SRS interactive targets: login and p95 concurrent requests remained below three seconds, dashboard loading remained below five seconds, and forecast generation remained below ten seconds. The result also satisfies the requirement to support 100 concurrent users without errors. Cold-start measurements were higher than warm requests but remained within the broader operational limits.

5.5 Recovery Verification

The stored machine-readable recovery evidence shows a successful local isolated drill that restored 1,055 documents across 16 collections in 13.914 seconds and removed the prefixed temporary collections after verification. A later scheduled production run recorded in the QA report restored 1,137 documents across 16 collections in 23.454 seconds. Both results are well within the ten-minute recovery objective.

Encryption and hash verification protect confidentiality and detect modified backup content. Because restoration uses temporary collection prefixes, the workflow verifies readability and document counts without

replacing the live database. This makes the recovery check safe to run as an operational control rather than leaving recovery as an untested assumption.

5.6 Presentation Workspace

The production demonstration workspace contains eight products, 800 sales records, a complete 30-day forecast, three downloadable reports, fourteen notifications and four replenishment recommendations. This provides a stable end-to-end path for the internal defence: sign in, review the dashboard, inspect products, upload or validate data, run a forecast, examine the selected model, review an order recommendation and open the same workspace on mobile.

5.7 Requirements Compliance

All functional requirements REQ-1 through REQ-18 have implementation evidence. Performance requirements were validated with timed production checks. Security requirements are covered by password hashing, JWT sessions, permission middleware, tenant filters and HTTPS. Reliability requirements are supported by controlled errors, health monitoring and verified recovery. Usability is supported by responsive navigation, explicit validation messages and a fifteen-minute user guide.

5.8 Discussion

The strongest project result is the connection between predictive modelling and an operational decision. A stored 30-day sequence supports graphs and reports, but the replenishment calculation makes the output directly useful. The recommendation lifecycle also recognises that a model advises rather than automatically purchases; a business user remains responsible for approval and can dismiss a recommendation when contextual information is missing from the data.

The hybrid model architecture is another important result. The Python service provides the three algorithms promised in the proposal. The imported XGBoost bundle adds a high-volume global candidate backed by recursive validation, and the seasonal model provides resilience for short or unmapped series. Candidate-level failure handling prevents one unavailable algorithm from crashing the entire workflow.

Measured performance shows that the application can satisfy its target workload in the deployed environment. The mobile client and web client use the same API, which reduces duplicated business rules. Tenant-scoped queries and auditable permission checks make the SaaS structure defensible beyond the interface layer.

5.9 Limitations

- No forecasting system can compensate for incorrect dates, missing products or sales that do not represent normal operations.
- Promotions, holidays, supplier reliability, weather and economic variables are not yet included as external features.
- The global XGBoost model is strongest for identifiers represented by its training mappings; new series need sufficient local history.
- SMTP email and public app-store delivery depend on credentials and approvals outside the submitted repository.
- Long-term availability statistics require monitoring over a longer production period than the development schedule.

CHAPTER 6
SUMMARY, CONTRIBUTIONS AND CONCLUSION

6.1 Summary

StockGuard AI was designed to help SMEs replace manual demand estimates with an integrated forecasting and inventory decision process. The final prototype includes secure identity, multi-user roles, product and sales management, strict CSV ingestion, five forecasting paths, 7/14/28/30-day horizons, revenue estimation, replenishment recommendations, dashboards, reports, insights, notifications, administration and web/mobile access.

The system was deployed and verified against the SRS. Model evidence includes a three-fold recursive XGBoost backtest over 986,433 observations. Application evidence includes authenticated integration checks, a 100-user production load test, complete thirty-point forecast persistence and encrypted isolated recovery.

6.2 Main Findings

1. The global XGBoost model reduced MAE by 12.72 percent relative to the seasonal baseline across the combined recursive backtest.
2. Live per-series comparison can evaluate Linear Regression, ARIMA and Random Forest and can include XGBoost and seasonal candidates when eligible.
3. Forecast totals can be converted into lead-time demand, safety stock, target stock and a concrete order quantity without manual spreadsheet formulas.
4. The deployed API served 100 authenticated virtual users with zero errors and p95 response time of 832 ms.
5. Encrypted backup artifacts can be restored and verified in isolated collections far within the ten-minute recovery objective.

6.3 Contributions

The project contributes a complete SME-oriented forecasting workflow rather than a model demonstration alone. It integrates model comparison with persistent daily forecasts, revenue calculations and replenishment actions. It also contributes a consistent web/mobile experience backed by one tenant-isolated API, reducing behavioural differences across platforms.

From an engineering perspective, the project provides traceability from SRS requirements to routes, schemas, interfaces and machine-readable evidence. The model bundle is protected by a recorded SHA-256 value; backup artifacts use authenticated encryption; and administrative activity is recorded for later review.

6.4 Recommendations

- SMEs should begin with at least twelve months of clean product-level sales data where available and review outliers before forecasting.
- Users should maintain realistic supplier lead times, on-hand quantities and reserved stock so order recommendations reflect operations.
- Forecast metrics should be reviewed per product; a single preferred algorithm should not be forced across all series.
- High-risk recommendations should be reviewed promptly, but approval should consider promotions, supplier constraints and business context not present in the model.
- Production operators should keep scheduled health, backup and restore verification workflows active and rotate secrets periodically.

6.5 Future Work

Future work can add promotions, holidays, planned price changes, supplier reliability, weather and economic indicators as model inputs. Hierarchical forecasting could reconcile product, category, store and total-

business estimates. Prediction intervals could communicate uncertainty directly, and model drift monitoring could trigger retraining based on measured degradation rather than a fixed schedule.

Operational extensions can include direct integrations with point-of-sale and accounting platforms, supplier purchase-order workflows, offline mobile caching and multilingual guidance. Public mobile distribution can be completed after Apple and Google developer approvals. MongoDB point-in-time recovery can complement the existing encrypted daily backup when the production plan supports it.

6.6 Conclusion

The completed StockGuard AI prototype meets the central commitments of the approved proposal and SRS. It accepts business sales history, compares forecasting models, stores complete future predictions and produces interpretable revenue and replenishment guidance. Its dashboards and mobile application make the analysis accessible, while JWT sessions, permissions, tenant isolation and audit logs protect multi-user business data.

The measured model, application and recovery results show that the design is technically viable for an SME prototype. Most significantly, the system turns a historical CSV file into a traceable purchasing recommendation in one workflow. This connection between prediction, business action and verified software behaviour is the principal outcome of the project.

REFERENCES

[1] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 2nd ed., OTexts, 2018.

[2] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

[3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed., Wiley, 2015.

[4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[6] G. Shmueli and O. R. Koppius, “Predictive Analytics in Information Systems Research,” MIS Quarterly, vol. 35, no. 3, pp. 553–572, 2011.

[7] R. H. Ballou, Business Logistics/Supply Chain Management, 5th ed., Pearson, 2004.

[8] C. C. Holt, “Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages,” International Journal of Forecasting, vol. 20, no. 1, pp. 5–10, 2004 (original memorandum 1957).

[9] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006. (Referenced for systematic software translation and validation concepts.)

[10] OWASP Foundation, OWASP Application Security Verification Standard 4.0.3, 2021.

[11] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.

[12] MongoDB Inc., MongoDB Manual: Data Modeling, Indexes and Security, product documentation, accessed 2026.

[13] Meta Platforms, React and React Native Documentation, product documentation, accessed 2026.

[14] OpenJS Foundation, Node.js Documentation, product documentation, accessed 2026.

[15] I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

[16] NIST, Digital Identity Guidelines: Authentication and Lifecycle Management, SP 800-63B, updated guidance.

CHAPTER A
SRS REQUIREMENT COMPLIANCE

The table below summarises the final implementation status of the eighteen functional requirements submitted in the StockGuard AI SRS. Detailed route and test evidence is retained in docs/SRS_COMPLIANCE.md in the project repository.

Table 11: SRS requirement compliance

Requirement	Description	Status
REQ-1	Registration and validation	Complete
REQ-2	JWT login and dashboard access	Complete
REQ-3	Protected business data	Complete
REQ-4	10 MB / 10,000-row CSV validation	Complete
REQ-5	Product demand forecast	Complete
REQ-6	LR, ARIMA and Random Forest	Complete
REQ-7	MAE/RMSE comparison and selection	Complete
REQ-8	Authorized forecast access	Complete
REQ-9	Product and sales management	Complete
REQ-10	Revenue and replenishment insight	Complete
REQ-11	Responsive analytics	Complete
REQ-12	Upload, forecast and critical alerts	Complete
REQ-13	Administration and activity monitoring	Complete
REQ-14	Audit retention	Complete — one year
REQ-15	Role-based access	Complete
REQ-16	Graceful errors	Complete
REQ-17	Daily backup and recovery	Complete
REQ-18	Privacy and secure communication	Complete

CHAPTER B
VERIFICATION EVIDENCE

The following evidence is reproducible from repository scripts and the deployed application. Test users and verification records are created in isolated tenants and are removed when the check completes.

Verification	Evidence / result
Backend static checks	Route, model, permission and forecasting assertions passed
Authentication integration	Register, login, refresh, logout, revocation and password reset passed
Forecast integration	Thirty daily predictions and candidate metrics saved
Analytics integration	Dashboard aggregations and tenant isolation passed
Web build	Vite production build passed
Mobile check	TypeScript check and Android production export passed
Concurrent production load	100/100 HTTP 200; average 259 ms; p95 832 ms
Recovery	16 isolated collections restored and verified; temporary data removed

CHAPTER C
PRINCIPAL REST API ENDPOINTS

Table 12: Principal REST API endpoints

Method	Path	Purpose
POST	/api/auth/register	Create user and isolated business workspace
POST	/api/auth/login	Authenticate and issue access/refresh session
POST	/api/auth/refresh	Rotate or renew session
GET/PATCH	/api/auth/me	Read or update authenticated profile
GET/POST	/api/products	List or create workspace products
PATCH/DELETE	/api/products/:id	Edit or remove an authorized product
POST	/api/uploads/validate	Validate CSV before commit
POST	/api/uploads	Import a validated sales batch
POST	/api/forecasts/run	Run and save a product/store forecast
GET	/api/forecasts	List saved forecast runs
GET	/api/recommendations	List replenishment actions
PATCH	/api/recommendations/:id/status	Approve, dismiss, complete or reopen
GET	/api/dashboard	Return tenant analytics
GET/POST	/api/reports	List or generate business reports
GET	/api/notifications	List persistent user notifications
GET/PATCH	/api/settings	Read or update business preferences
GET/PATCH	/api/admin/users	Manage users, roles and permissions
GET	/api/admin/audit	Review tenant activity
GET	/api/health	Read production service health

CHAPTER D
INTERNAL DEFENCE DEMONSTRATION SEQUENCE

1. Open the production URL and sign in to the prepared StockGuard AI demonstration workspace.
2. Use the dashboard to explain total products, sales activity, forecast status, stock risks and current recommendations.
3. Open Products and demonstrate product details, inventory quantities, supplier lead time and an edit action.
4. Open CSV Upload, explain the required columns, 10 MB and 10,000-row constraints, and validation feedback.
5. Open AI Forecast, choose a product/store series, select 30 days and run the candidate comparison.
6. Explain MAE, RMSE, the selected model, daily predictions, total demand and forecast revenue.
7. Open the replenishment recommendation and explain lead-time demand, safety stock, target stock and recommended order quantity.
8. Change a recommendation status to demonstrate the approval workflow and audit trail.
9. Open Reports and Insights to show persisted results and business interpretation.
10. Open the mobile application and show that the same tenant data, forecasts and recommendations are available.
11. Finish with Admin, Notifications, Settings and the health/backup evidence to demonstrate operational completeness.

Defence Evidence Summary

Claim	Evidence to show
Forecasting is live	Run a 30-day forecast and inspect all thirty saved daily points
Models are compared	Show LR, ARIMA, RF and eligible production candidates with MAE/RMSE
Inventory advice is calculated	Show safety stock, target stock and order quantity
Web and mobile share data	Open the same product/forecast in both clients
Security is enforced	Show roles, permissions and audit records
System is production-ready	Show health response, load evidence and recovery result

CHAPTER E
SUBMISSION AND DEPLOYMENT CHECKLIST

- Confirm the live web URL opens and the API health endpoint reports MongoDB, Python ML and XGBoost readiness.
- Keep the prepared presentation user credentials private and test login before the defence.
- Open the mobile project through Expo Go on the same network or use a prepared preview build.
- Keep the model bundle, manifest and category mappings in the deployed server package.
- Verify the default forecast horizon is 30 days and that settings are saved for the demonstration workspace.
- Confirm the encrypted backup workflow and production-health workflow show successful recent runs.
- Attach the official similarity report and obtain the required signatures on certificate pages.
- Submit this report together with the approved proposal, SRS, source repository and presentation materials.